

INF2007 – TN4 – Melissa Moya

1. Approche et structure du programme

Le programme calcule la somme des sinus d'un tableau de 1 000 000 d'éléments (entiers ou flottants) via le flag `--type`. Deux fonctions spécialisées (`computeSineSumInt` / `computeSineSumFloat`) contiennent la boucle de calcul, tandis que `computeSineSum` dispatche via `interface{}` avec un surcoût négligeable (~ 1 ns) face à `math.Sin` (~ 20 – 40 ns). Pour garantir la reproductibilité, les tableaux sont générés avec `rand.NewSource(42)`. Bien que `rand.Intn(1001)` introduise un léger biais pour les plages non puissances de deux, celui-ci reste négligeable, car l'objectif est d'obtenir des données représentatives, non une distribution parfaitement uniforme. Côté architecture, `run()` retourne un struct `RunResult` sans affichage et `main()` gère seule la présentation des résultats, assurant une séparation nette entre logique et affichage.

Résultats complets, benchmarks et captures : [docs/](#) · [logs/](#)

2. Résultats des benchmarks

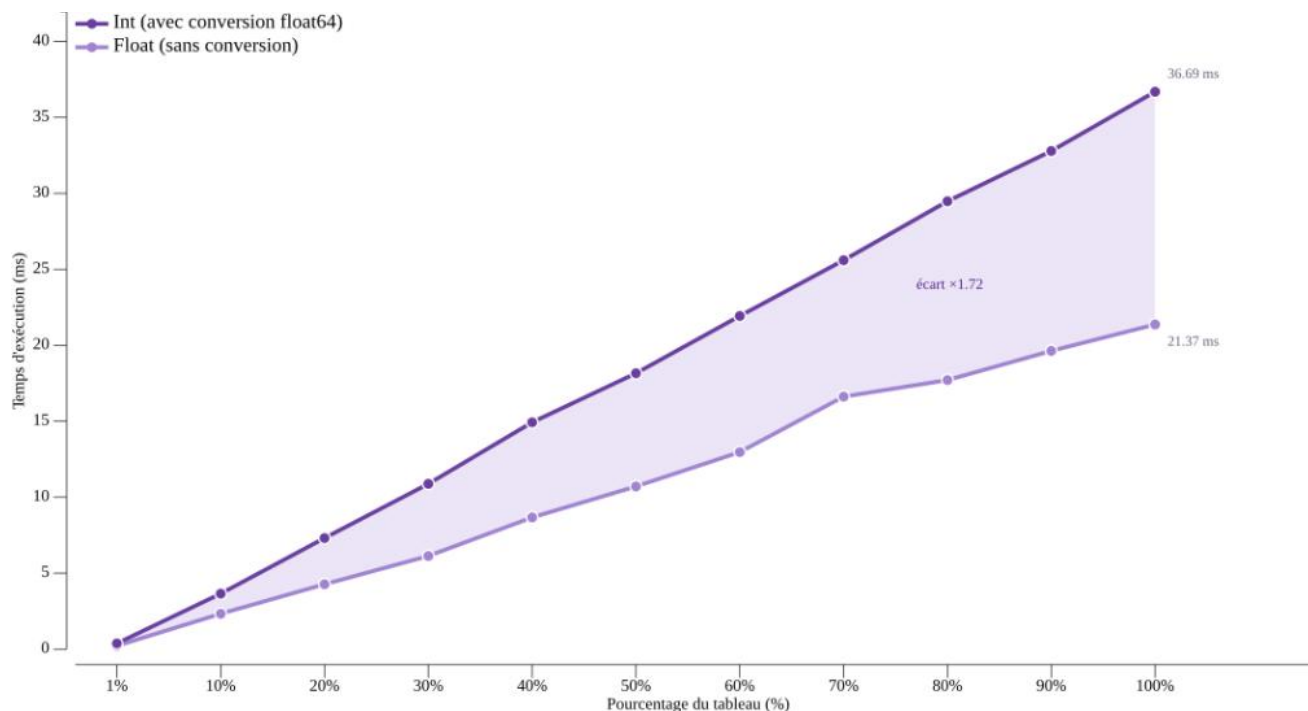
Les 22 sous-benchmarks (11 paliers $\times$ 2 types) ont été exécutés avec `go test -bench=. -benchmem -count=6` sur un Intel i5-10300H à 2,50 GHz (Windows/amd64, 8 threads) et analysés avec `benchstat` (médianes + IC 95 %). Pour isoler le pur coût de calcul, `b.ResetTimer()` exclut le setup, les tableaux sont pré-générés hors de `b.N`, et `b.ReportAllocs()` confirme 0 B/op, 0 allocs/op, ce qui indique que la performance est entièrement liée au calcul, sans aucune allocation mémoire. Le projet compte par ailleurs 13 tests unitaires avec une couverture de 100 %. Les résultats présentés ci-dessous sont issus d'une nouvelle exécution fraîche (183 s) afin de garantir des mesures à jour.

Tableau 1 – Temps de calcul par type et pourcentage du tableau

% du tableau	Éléments	Int (ms)	Float (ms)	Ratio
1 %	10 000	0.376	0.218	1.73×
10 %	100 000	3.65	2.33	1.57×
20 %	200 000	7.31	4.27	1.71×
30 %	300 000	10.88	6.13	1.77×
40 %	400 000	14.93	8.67	1.72×
50 %	500 000	18.16	10.71	1.70×
60 %	600 000	21.93	12.97	1.69×
70 %	700 000	25.60	16.62 ± 50 %*	1.54×
80 %	800 000	29.48	17.71	1.66×
90 %	900 000	32.79	19.63	1.67×
100 %	1 000 000	36.69	21.37	1.72×

Les flottants sont systématiquement plus rapides, avec un ratio de 1,5 à 1,8× selon le palier. De 50 % à 100 %, le temps double presque exactement (18,16 $\rightarrow$ 36,69 ms pour Int ; 10,71 $\rightarrow$ 21,37 ms pour Float), ce qui confirme bien la complexité $O(n)$. Aux paliers 90–100 %, l'intervalle de confiance atteint ± 1 %, signe de mesures stables. *Le palier 70 % Float présente un IC de ± 50 %, attribuable à une perturbation système ponctuelle lors de cette exécution.

Graphique 1 – Temps de calcul selon le pourcentage du tableau (Int vs Float)



La zone mauve translucide représente l'écart Int – Float ; l'annotation « $1.72\times$ » indique le ratio à 100 %. Pour régénérer le graphique, voir docs/chart-guide.md.

3. Analyse des résultats

Complexité et écart Int/Float

Les flottants sont 1,5 à 1,8× plus rapides. L'écart provient de la conversion float64(v) à chaque itération et du coût de réduction d'argument de math.Sin pour les grands entiers, notamment $\sin(500)$ réduit ~ 79 tours modulo 2π , alors que les flottants dans $[0, 1)$ se situent déjà dans le domaine principal et ne requièrent aucune réduction. La variabilité exceptionnelle au palier 70 % (Float ± 50 %) est une valeur atypique due à une perturbation système ponctuelle lors de cette exécution ; les autres paliers Float restent stables (± 3 –13 %). Le palier 30 % Float (± 3 %) est désormais bien plus régulier que lors de la précédente exécution.

Hierarchie mémoire — L1/L2/L3

L'i5-10300H dispose de L1 = 256 KB, L2 = 1 MB, L3 = 8 MB. Les petits paliers (1–10 %, 80–800 KB) tiennent entièrement en L2 ; le palier 30 % ($\sim 2,4$ MB) dépasse le L2 et sollicite le L3 ; le tableau complet ($1\,000\,000 \times 8$ octets = 8 MB) atteint exactement la limite du L3, limitant les accès par la bande passante, ce qui explique les légères non-linéarités aux paliers 70 %+.

Mécanismes micro-architecturaux

go build -gcflags=-m confirme que math.Sin est inliné aux lignes 40 et 49 de `sinesum.go`, éliminant le surcoût d'appel de fonction. La boucle `sum += math.Sin(v)` crée une dépendance de données sur

l'accumulateur sum : chaque addition doit attendre le résultat précédent, ce qui empêche le pipeline FPU de superposer les opérations (pas de pipelining des additions), identique à l'exemple prefixSum du chapitre 6. Les appels math.Sin(v) sur des v indépendants sont eux exécutés en désordre (out-of-order execution), mais ce gain est plafonné par la latence intrinsèque de math.Sin (~20–40 ns), bien supérieure à son débit théorique. Les intervalles de confiance benchstat confirment la stabilité globale : Int/100pct affiche ± 1 %, les paliers Int restant tous dans ± 6 %. La principale exception est Float/70pct (± 50 %), valeur atypique liée à une perturbation système ponctuelle. Utiliser plusieurs accumulateurs fusionnés en fin de calcul réduirait la dépendance de données et constituerait la principale piste d'optimisation.

4. Applications numériques

En prenant les médianes benchstat à 100 % du tableau, on obtient un temps par sinus de 36,7 ns pour les entiers (36 690 000 ns $\div$ 1 000 000) et de 21,4 ns pour les flottants (21 370 000 ns $\div$ 1 000 000).

Q1 — Distance parcourue par la lumière pendant un sinus ($c = 299\,792\,458$ m/s)

Type	Temps	Distance
Int	36,7 ns	$299,792,458 \times \frac{36,7}{10^9} \approx 11,0$ m
Float	21,4 ns	$299,792,458 \times \frac{21,4}{10^9} \approx 6,4$ m

Réponse : Pendant le calcul d'un sinus, la lumière parcourt environ 11,0 m avec des entiers et 6,4 m avec des flottants.

Q2 — Nombre de sinus par tick à 120 fps (tick = $\frac{1}{120}$ s = 8 333 333 ns)

Type	Temps/sinus	Sinus/tick
Int	36,7 ns	$8,333,333 \div 36,7 \approx 227\,065$
Float	21,4 ns	$8,333,333 \div 21,4 \approx 389\,408$

Réponse : À 120 fps, on peut calculer environ 227 000 sinus par frame avec des entiers et 389 000 avec des flottants.

Détails des calculs : [calculs.md](#)